

Data-Driven Physical Modelling Coursework Part 2

Notes:

- Make sure the data file is in the same directory as the .ipynb file
- Below are the following packages used in the entirety of Coursework Part 2 including the seeds
- Written times in results for codes that took long to run

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import random

seed = 15

random.seed(seed)
np.random.seed(seed)
torch.manual_seed(seed)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(seed)
```

Exploratory Data Analysis

Question 1

```
data = np.load('vorticity_vary_Re_U.npz')

# Extract keys
Re = data['Re']
U = data['U']
radii = data['radii']
sigma = data['sigma_mean']

# Number of simulations
N = Re.shape[0]

split_idx = int(0.8 * N)

# Create shuffled indices under seed randomisation
indices = np.random.permutation(N)

train_idx = indices[:split_idx]
test_idx = indices[split_idx:]

# Build train/test splits
```

```

train_data = {
    "Re": Re[train_idx],
    "U": U[train_idx],
    "radii": radii[:, train_idx],
    "sigma_mean": sigma[:, :, :, train_idx]
}

test_data = {
    "Re": Re[test_idx],
    "U": U[test_idx],
    "radii": radii[:, test_idx],
    "sigma_mean": sigma[:, :, :, test_idx]
}

```

The code above uses a trajectory split instead of a column split. A column split would cause different spatial locations from the same flow in the training and the testing dataset, causing data leakage. On the other hand, a trajectory split assigns complete simulations to the testing or training sets, allowing the model to be evaluated on new flow trajectories instead of spatial interpolation on already-seen flows.

Question 2

```

# Extract for Training Data
train_ts_9_7 = train_data["sigma_mean"][9, 7, :, :]
train_ts_10_7 = train_data["sigma_mean"][10, 7, :, :]

# Extract for Testing Data
test_ts_9_7 = test_data["sigma_mean"][9, 7, :, :]
test_ts_10_7 = test_data["sigma_mean"][10, 7, :, :]

# All the above data are put into dictionaries for easy access later on
train_data["ts_9_7"] = train_ts_9_7
train_data["ts_10_7"] = train_ts_10_7

test_data["ts_9_7"] = test_ts_9_7
test_data["ts_10_7"] = test_ts_10_7

```

Question 3

```

# Mean and standard deviation for z score standardisation
def get_stats(array):
    return np.mean(array), np.std(array)

# Apply z score standardisation
def apply_norm(array, mean, std):
    return (array - mean) / (std + 1e-8)

```

```

# 1. Compute statistics on train
mu_Re,      std_Re      = get_stats(train_data["Re"])
mu_U,       std_U       = get_stats(train_data["U"])
mu_ts_9_7,  std_ts_9_7  = get_stats(train_data["ts_9_7"])
mu_ts_10_7, std_ts_10_7 = get_stats(train_data["ts_10_7"])

# 2. Standardise training data
train_data["Re_norm"]      = apply_norm(train_data["Re"],
                                         mu_Re,      std_Re)

train_data["U_norm"]       = apply_norm(train_data["U"],
                                         mu_U,       std_U)

train_data["ts_9_7_norm"]  = apply_norm(train_data["ts_9_7"],
                                         mu_ts_9_7,  std_ts_9_7)

train_data["ts_10_7_norm"] = apply_norm(train_data["ts_10_7"],
                                         mu_ts_10_7, std_ts_10_7)

# 3. Apply training statistics to standardise test data
test_data["Re_norm"]       = apply_norm(test_data["Re"],
                                         mu_Re,      std_Re)

test_data["U_norm"]        = apply_norm(test_data["U"],
                                         mu_U,       std_U)

test_data["ts_9_7_norm"]   = apply_norm(test_data["ts_9_7"],
                                         mu_ts_9_7,  std_ts_9_7)

test_data["ts_10_7_norm"]  = apply_norm(test_data["ts_10_7"],
                                         mu_ts_10_7, std_ts_10_7)

```

This code uses z score standardisation to standardise the data extracted from the original dataset using

$$x_{norm} = \frac{x - \mu}{\sigma}$$

. This allows all the variables to be on comparable scales since they all have mean 0 and standard deviation 1 from the training set. No single variable is able to distort any results due to a generally high magnitude compared to other variables. The signs are still preserved so the oscillatory behaviour is still retained as the z score is applied. Using the train statistics on the test data allows the test data to have no effect on the learning of the standardisation, preventing leakage.

Question 4

```

# Build time axis
T = sigma.shape[2]

```

```

dt = 0.1
time = np.arange(T) * dt

# Function for similar Re, varying U
def choose_three_similar_Re_diff_U(Re, U, re_tol=25, min_u_diff=0.35):

    # sort by Re and pick a central Reynolds number
    idx_sorted_Re = np.argsort(Re)
    idx_center = idx_sorted_Re[len(Re) // 2]
    re_target = Re[idx_center]

    # candidates with Re close to re_target
    idxs = np.where(np.abs(Re - re_target) < re_tol)[0]
    if len(idxs) < 3:
        raise RuntimeError("Not enough simulations with similar Re,
increase re_tol.")

    # Sort indices by U
    idxs_sorted_by_U = idxs[np.argsort(U[idxs])]

    # Pick greatly differing U values
    i_low = idxs_sorted_by_U[0]
    i_mid = idxs_sorted_by_U[len(idxs_sorted_by_U) // 2]
    i_high = idxs_sorted_by_U[-1]
    sel = [i_low, i_mid, i_high]

    # Optional check to make sure U values are sufficiently different
    u_vals = U[sel]
    if (
        np.min(
            np.abs(u_vals[:, None] - u_vals[None, :]) + np.eye(3) *
1e9
        )
        < min_u_diff
    ):
        print("Warning: chosen U values are closer than min_u_diff")

    return sel

idxs_Re_fixed = choose_three_similar_Re_diff_U(Re, U)

# Function for similar U, varying Re
def choose_three_similar_U_diff_Re(Re, U, u_tol=0.01,
min_re_diff=200.0):

    # Sort by U and pick a central flow speed
    idx_sorted_U = np.argsort(U)
    idx_center = idx_sorted_U[len(U) // 2]

```

```

u_target = U[idx_center]

# Candidates with U close to actual U
idxs = np.where(np.abs(U - u_target) < u_tol)[0]
if len(idxs) < 3:
    raise RuntimeError("Not enough simulations with similar U," \
        " increase u_tol.")

# Sort candidates by Re
idxs_sorted_by_Re = idxs[np.argsort(Re[idxs])]

# Pick greatly differing Re values
i_low = idxs_sorted_by_Re[0]
i_mid = idxs_sorted_by_Re[len(idxs_sorted_by_Re) // 2]
i_high = idxs_sorted_by_Re[-1]
sel = [i_low, i_mid, i_high]

return sel

idxs_U_fixed = choose_three_similar_U_diff_Re(Re, U)

# Plots for (9,7) and (10,7) for all varying Re and U
fig, axs = plt.subplots(2, 2, figsize=(14, 10))

# Similar Re, varying U at (9,7)
ax = axs[0, 0]
for idx in idxs_Re_fixed:
    ax.plot(
        time,
        sigma[9, 7, :, idx],
        label=f"Re={Re[idx]:.1f}, U={U[idx]:.2f}",
    )
ax.set_title("Similar Re, varying U - vorticity at spatial coordinate (9,7)")
ax.set_xlabel("Time/s")
ax.set_ylabel("Vorticity")
ax.legend()

# Similar Re, varying U at (10,7)
ax = axs[0, 1]
for idx in idxs_Re_fixed:
    ax.plot(
        time,
        sigma[10, 7, :, idx],
        label=f"Re={Re[idx]:.1f}, U={U[idx]:.2f}",
    )
ax.set_title("Similar Re, varying U - vorticity at spatial coordinate (10,7)")
ax.set_xlabel("Time/s")

```

```

ax.set_ylabel("Vorticity")
ax.legend()

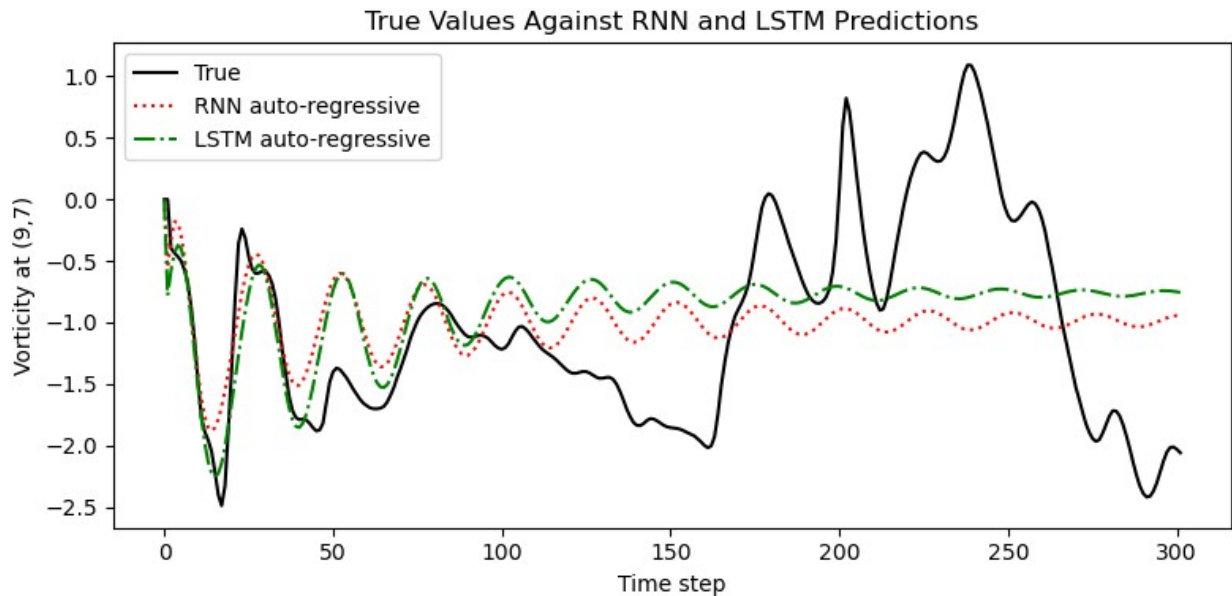
# Similar U, varying Re at (9,7)
ax = axs[1, 0]
for idx in idxs_U_fixed:
    ax.plot(
        time,
        sigma[9, 7, :, idx],
        label=f"Re={Re[idx]:.1f}, U={U[idx]:.2f}",
    )
ax.set_title("Similar U, varying Re - vorticity at spatial coordinate (9,7)")
ax.set_xlabel("Time/s")
ax.set_ylabel("Vorticity")
ax.legend()

# Similar U, varying Re at (10,7)
ax = axs[1, 1]
for idx in idxs_U_fixed:
    ax.plot(
        time,
        sigma[10, 7, :, idx],
        label=f"Re={Re[idx]:.1f}, U={U[idx]:.2f}",
    )
ax.set_title("Similar U, varying Re - vorticity at spatial coordinate (10,7)")
ax.set_xlabel("Time/s")
ax.set_ylabel("Vorticity")
ax.legend()

plt.tight_layout()
plt.show()

```

Results:



a) When looking at the short term time period, the three curves on every graph are very close to each other. Even when Re differs by a factor of 11 between simulations, or U changes by a factor of 4, the vorticity in the early stages remains almost identical across simulations, indicating that the initial flow behaviour is not strongly affected by Re or U .

b) Over the full time series, the effect of the varying Re and U values becomes much more prevalent. At both $(9,7)$ and $(10,7)$, as U increases, the simulations experience larger vorticity peaks and troughs, as shown by the green lines compared to the orange and the blue lines. The same can be said about the varying Re values, where it exhibits an even stronger effect on the vorticity. The lower the Re , the smoother the curve is than the greater values of Re at both $(9,7)$ and $(10,7)$.

Question 5

```
ny, nx, T, N_sims = sigma.shape
n_features = ny * nx

# Array to store number of PCs needed for 99.5 % variance at each time
n_pc = np.zeros(T, dtype=int)

for t in range(T):
    # Extract vorticity field at time t across all simulations
    # Shape before reshape: (ny, nx, N_sims)
    field_t = sigma[:, :, t, :]

    # Reshaping to get the same shape as n_features
    X_t = field_t.reshape(ny * nx, N_sims).T

    # Total variance across all features, same definition sklearn uses
    total_var = np.var(X_t, axis=0, ddof=1).sum()
```

```

# Filtering out invalid cases
if total_var < 1e-12:
    n_pc[t] = 0
    continue

# PCA for centring features
pca = PCA()
pca.fit(X_t)

# Cumulative explained variance ratio
cum_var = np.cumsum(pca.explained_variance_ratio_)

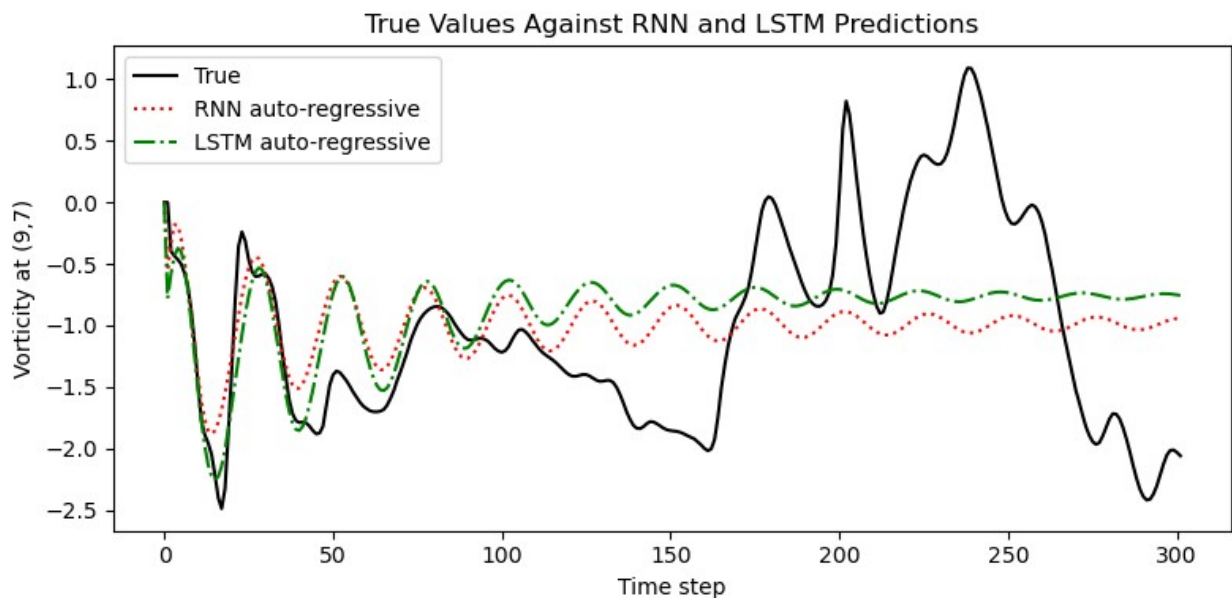
# Smallest k where cumulative variance >= 0.995
k = np.searchsorted(cum_var, 0.995) + 1
n_pc[t] = k

# Plot PCs vs time
time = np.arange(T) * 0.1

plt.figure(figsize=(8, 4))
plt.plot(time, n_pc, marker="o", linewidth=1)
plt.xlabel("Time/s")
plt.ylabel("Number of PCs")
plt.title("Number of PCs to capture 99.5% variance against time")
plt.grid(True)
plt.show()

```

Results:



At the start, very few components are required to capture 99.5% of the variance. This shows that the simulations are very similar and most of the variability lies along a single spatial pattern,

which matches what was seen in question 4 where the curves were generally on top of each other in the short term horizon. As time goes on, the number of PCs increases steadily, reaching a maximum of 175 at the end of the simulation. This shows that the spatial structure of the vorticity across the simulations are getting more complex and so the variance is spread across more principle components. Since the maximum number of principle components required to cover 99.5% of the variance is 175, while the spatial constraints is 384, the data lies in a subspace of at most 175 dimensions within the original 384 dimensions.

Echo State Networks

Question 1

```
def create_res(size, in_size=1, spectral_radius=0.9,
input_scaling=1.0):

    # Random reservoir in [-0.5, 0.5]
    W = np.random.rand(size, size) - 0.5

    # Diagonal set to 0 so no self feedback
    np.fill_diagonal(W, 0.0)

    # Scale to desired spectral radius
    eigvals = np.linalg.eigvals(W)

    # Obtain maximum eigenvalue magnitude
    max_eig = np.max(np.abs(eigvals))
    W *= spectral_radius / max_eig

    # Random input weights in [-0.5, 0.5] then scale by input_scaling
    Win = np.random.rand(size, in_size) - 0.5
    Win *= input_scaling

    return W, Win

# Takes input sequence and returns reservoir states
def run_res(W, Win, u, x=None, leak_rate=1.0):
    if x is None:
        x = np.zeros(W.shape[0])

    T = u.shape[0]
    all_x = np.zeros((T, W.shape[0]))

    alpha = leak_rate

    for t in range(T):
        pre_act = W @ x + Win @ u[t, :]
        x = (1 - alpha) * x + alpha * np.tanh(pre_act)
        all_x[t, :] = x

    return all_x
```

```

# Output fitted using regularisation parametric
def train_res(all_x, y, beta):
    N = all_x.shape[1]
    A = all_x.T @ all_x + beta * np.eye(N)
    B = all_x.T @ y
    Wout = np.linalg.solve(A, B)
    return Wout

# Trained ESN predicts outputs
def predict_res(Wout, W, Win, u, leak_rate=1.0):
    x = np.zeros(W.shape[0])
    T = u.shape[0]
    n_outputs = Wout.shape[1]
    y_pred = np.zeros((T, n_outputs))

    alpha = leak_rate

    for t in range(T):
        pre_act = W @ x + Win @ u[t, :]
        x = (1 - alpha) * x + alpha * np.tanh(pre_act)
        y_pred[t, :] = (Wout.T @ x)

    return y_pred

# Normalised time series at (9,7)
y_train_n = train_data["ts_9_7_norm"]
y_test_n = test_data["ts_9_7_norm"]

# Normalised inputs (Re, U)
Re_train_n = train_data["Re_norm"]
U_train_n = train_data["U_norm"]
Re_test_n = test_data["Re_norm"]
U_test_n = test_data["U_norm"]

T, n_train = y_train_n.shape
n_test = y_test_n.shape[1]

# Fixed hyperparameters for ESN
reservoir_size = 500      # N
spectral_radius = 0.9     # rho
input_scaling = 1.0       # sigma_in
log10_beta = -4.0         # log(regularisation)
beta = 10 ** log10_beta   # beta
leak_rate = 1.0           # alpha

# Discard set number of initial time steps to avoid initialisation
# problems
washout = 25

```

```

W, Win = create_res(
    reservoir_size,
    in_size=2,
    spectral_radius=spectral_radius,
    input_scaling=input_scaling,
)

# Building training data for ESN
all_X = []
all_Y = []

for i in range(n_train):
    # Constant in time input [Re_i, U_i] for this simulation
    u_i = np.column_stack([
        np.full(T, Re_train_n[i]),
        np.full(T, U_train_n[i]),
    ])

    # Run reservoir
    x_i = run_res(W, Win, u_i, leak_rate=leak_rate)

    # Collect states plus targets after washout
    all_X.append(x_i[washout:, :])
    all_Y.append(y_train_n[washout:, i:i+1])

X_train = np.vstack(all_X)
Y_train = np.vstack(all_Y)

# Train output weights with ridge regression
Wout = train_res(X_train, Y_train, beta=beta)

# Error calculation
def sequence_mse(y_true, y_pred, washout=0):
    """Averages MSE over time steps after washout."""
    return np.mean((y_true[washout:] - y_pred[washout:])**2)

# Training MSE over simulations
mse_train_list = []

for i in range(n_train):
    u_i = np.column_stack([
        np.full(T, Re_train_n[i]),
        np.full(T, U_train_n[i]),
    ])
    y_pred_i = predict_res(Wout, W, Win, u_i, leak_rate=leak_rate)[:0]

```

```

        mse_i = sequence_mse(y_train_n[:, i], y_pred_i, washout=washout)
        mse_train_list.append(mse_i)

mse_train = np.mean(mse_train_list)
print(f"Normalised training MSE: {mse_train:.3f}")

# Testing MSE over simulations
mse_test_list = []

for j in range(n_test):
    u_j = np.column_stack([
        np.full(T, Re_test_n[j]),
        np.full(T, U_test_n[j]),
    ])

    y_pred_j = predict_res(Wout, W, Win, u_j, leak_rate=leak_rate)[:,
0]
    mse_j = sequence_mse(y_test_n[:, j], y_pred_j, washout=washout)
    mse_test_list.append(mse_j)

mse_test = np.mean(mse_test_list)
print(f"Normalised testing MSE: {mse_test:.3f}")

# Pick one simulation from train and test to visualise
ex_train = 0
ex_test = 0

# Training example
u_tr = np.column_stack([
    np.full(T, Re_train_n[ex_train]),
    np.full(T, U_train_n[ex_train]),
])

# Testing example
u_te = np.column_stack([
    np.full(T, Re_test_n[ex_test]),
    np.full(T, U_test_n[ex_test]),
])

# True and predicted time series for training
y_tr_true_n = y_train_n[:, ex_train]
y_tr_pred_n = predict_res(Wout, W, Win, u_tr, leak_rate=leak_rate)[:,
0]

# standardisation reversed back to original data
y_tr_true = y_tr_true_n * std_ts_9_7 + mu_ts_9_7
y_tr_pred = y_tr_pred_n * std_ts_9_7 + mu_ts_9_7

# True and predicted time series for testing
y_te_true_n = y_test_n[:, ex_test]

```

```

y_te_pred_n = predict_res(Wout, W, Win, u_te, leak_rate=leak_rate)[:  
0]

# Standardisation reversed back to original data
y_te_true = y_te_true_n * std_ts_9_7 + mu_ts_9_7
y_te_pred = y_te_pred_n * std_ts_9_7 + mu_ts_9_7

# Combined figure
fig, axs = plt.subplots(1, 2, figsize=(12, 4), sharey=True)

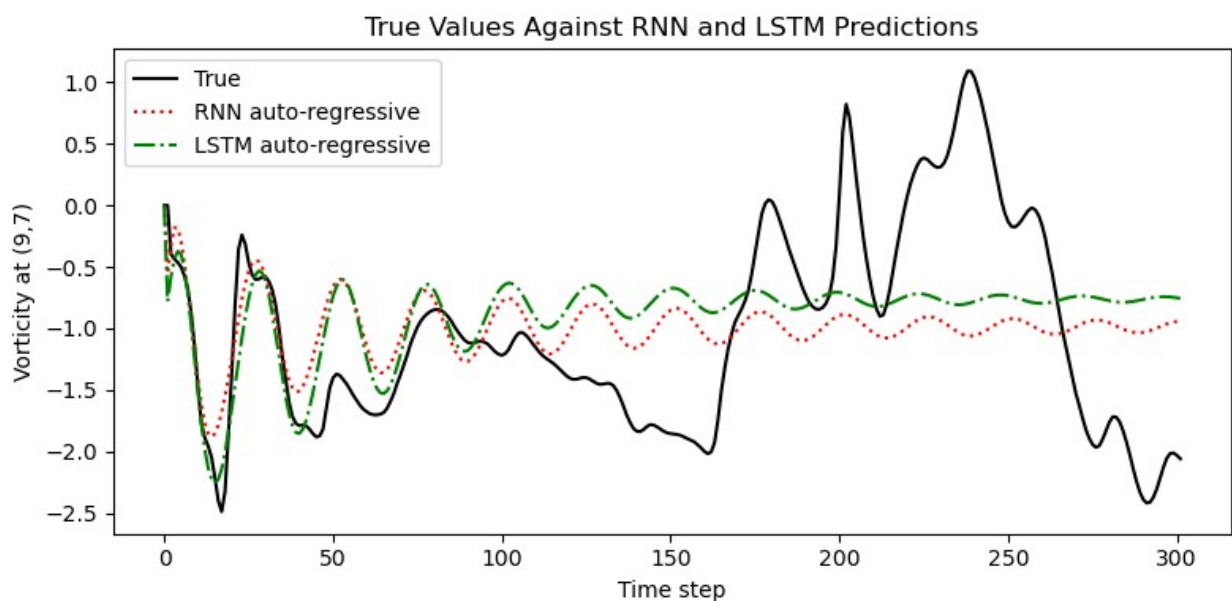
# Train subplot
axs[0].plot(y_tr_true, label="True (train)")
axs[0].plot(y_tr_pred, label="ESN prediction (train)")
axs[0].axvspan(0, washout, color="grey", alpha=0.2, label="washout")
axs[0].set_xlabel("time step")
axs[0].set_ylabel("vorticity at (9,7)")
axs[0].set_title("ESN prediction – training")
axs[0].legend(loc="best")

# Test subplot
axs[1].plot(y_te_true, label="True (test)")
axs[1].plot(y_te_pred, label="ESN prediction (test)")
axs[1].axvspan(0, washout, color="grey", alpha=0.2, label="washout")
axs[1].set_xlabel("time step")
axs[1].set_title("ESN prediction – test")
axs[1].legend(loc="best")

plt.tight_layout()
plt.show()

```

Results :



Normalised training MSE: 0.948

Normalised testing MSE: 0.937

The graph above shows the the ESN predictions with the test and train data. In the washout timeframe, there are huge spikes, particularly in the test data since the washout is at 20. The washout is the initial number of timesteps that are ignored by the reservoir so that the model mainly responds to the flow parameters. For the rest of the time series data, the ESN does not strongly overfit, as shown by the similar normalised training and testing MSEs of 0.948 and 0.937, respectively. However, the overall magnitudes of the MSEs show that there is poor performance.

There are two main reasons for the poor performance. The first is that the inputs are constant in time so the ESN will only predict a value of the vorticity based off the Re and U in the same timestep which does not give the trained model any time varying information about the vorticity. The second is that the ESN is predicting the vorticity with only two parameters. The dataset is a complex time series, so predicting vorticity based of only two parameters will be highly inaccurate.

Question 2

```
# Normalised input and target time series
u_train_n = train_data["ts_9_7_norm"]      # input output: vorticity at
(9,7)
y_train_n = train_data["ts_10_7_norm"]     # target output: vorticity
at (10,7)

u_test_n  = test_data["ts_9_7_norm"]       # testing input: vorticity
at (9,7)
y_test_n  = test_data["ts_10_7_norm"]     # testing output: vorticity
at (9,7)

T, n_train = u_train_n.shape
n_test = u_test_n.shape[1]

# Creates input vector of length 1
W_97_107, Win_97_107 = create_res(
    reservoir_size,
    in_size=1,
    spectral_radius=spectral_radius,
    input_scaling=input_scaling,
)

# Build training design matrix
all_X = []
all_Y = []

for i in range(n_train):
    u_i = u_train_n[:, i][:, np.newaxis]
    x_i = run_res(W_97_107, Win_97_107, u_i, leak_rate=leak_rate)
```

```

    all_X.append(x_i[washout:, :])
    all_Y.append(y_train_n[washout:, i:i+1])

X_train = np.vstack(all_X)
Y_train = np.vstack(all_Y)

# Train output weights with ridge regression
Wout_97_107 = train_res(X_train, Y_train, beta=beta)

# MSE over all time steps including washout
def full_mse(y_true, y_pred):
    return np.mean((y_true - y_pred)**2)

# Training MSE over simulations
mse_train_list = []
for i in range(n_train):
    u_i = u_train_n[:, i][:, np.newaxis]
    y_true_i = y_train_n[:, i]

    y_pred_i = predict_res(Wout_97_107, W_97_107, Win_97_107,
                           u_i, leak_rate=leak_rate)[: , 0]

    mse_i = full_mse(y_true_i, y_pred_i)
    mse_train_list.append(mse_i)

mse_train = np.mean(mse_train_list)
print(f"Normalised Training MSE: {mse_train:.3f}")

# Testing MSE over simulations
mse_test_list = []
for j in range(n_test):
    u_j = u_test_n[:, j][:, np.newaxis]
    y_true_j = y_test_n[:, j]

    y_pred_j = predict_res(Wout_97_107, W_97_107, Win_97_107,
                           u_j, leak_rate=leak_rate)[: , 0]
    mse_j = full_mse(y_true_j, y_pred_j)

    mse_test_list.append(mse_j)

mse_test = np.mean(mse_test_list)
print(f"Normalised Testing MSE: {mse_test:.3f}")

# Plot one of the training and testing simulations
ex_train = 0
ex_test = 0

u_tr = u_train_n[:, ex_train][:, np.newaxis]
y_tr_true_n = y_train_n[:, ex_train]

```

```

y_tr_pred_n = predict_res(Wout_97_107, W_97_107, Win_97_107,
                          u_tr, leak_rate=leak_rate)[: , 0]

y_tr_true = y_tr_true_n * std_ts_10_7 + mu_ts_10_7
y_tr_pred = y_tr_pred_n * std_ts_10_7 + mu_ts_10_7

u_te = u_test_n[:, ex_test][:, np.newaxis]
y_te_true_n = y_test_n[:, ex_test]

y_te_pred_n = predict_res(Wout_97_107, W_97_107, Win_97_107,
                          u_te, leak_rate=leak_rate)[: , 0]

y_te_true = y_te_true_n * std_ts_10_7 + mu_ts_10_7
y_te_pred = y_te_pred_n * std_ts_10_7 + mu_ts_10_7

# Plotting graph for both simulations
fig, axs = plt.subplots(1, 2, figsize=(12, 4), sharey=True)

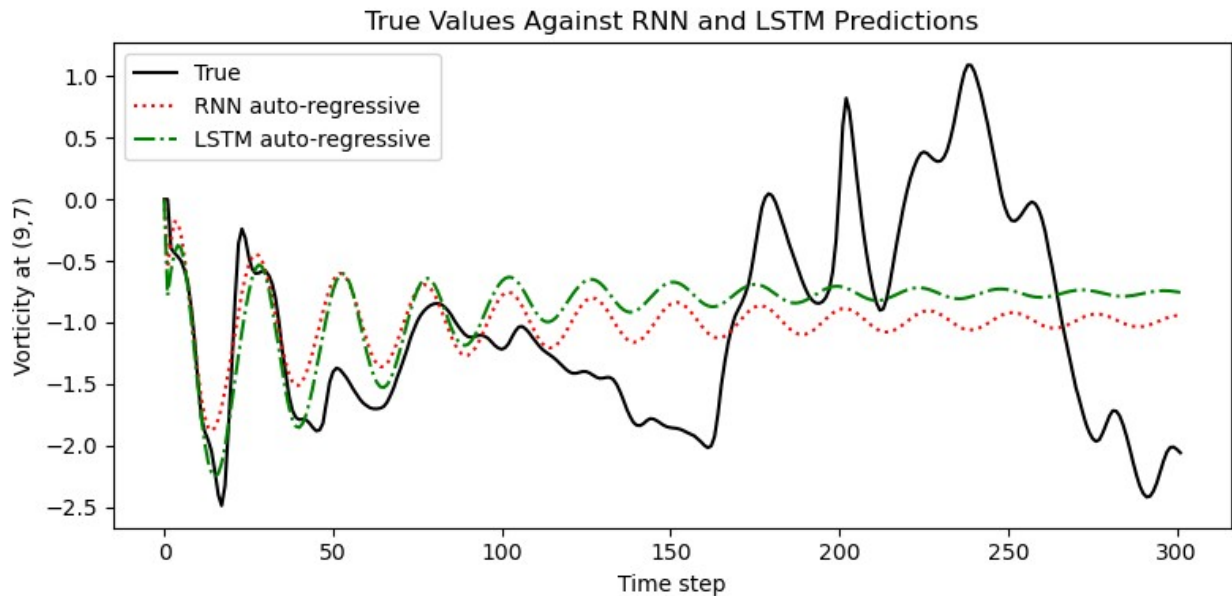
# Training
axs[0].plot(y_tr_true, label="True (train)")
axs[0].plot(y_tr_pred, label="ESN prediction (train)")
axs[0].set_xlabel("time step")
axs[0].set_ylabel("vorticity at (10,7)")
axs[0].set_title("Training ESN prediction at (10,7)")
axs[0].legend(loc="best")

# Testing
axs[1].plot(y_te_true, label="True (test)")
axs[1].plot(y_te_pred, label="ESN prediction (test)")
axs[1].set_xlabel("time step")
axs[1].set_title("Testing ESN prediction at (10,7)")
axs[1].legend(loc="best")

plt.tight_layout()
plt.show()

```

Results:



Normalised Training MSE: 3.919

Normalised Testing MSE: 3.962

Question 3

```
# Normalised input and target time series
u_train_n = train_data["ts_9_7_norm"]
y_train_n = train_data["ts_10_7_norm"]
u_test_n = test_data["ts_9_7_norm"]
y_test_n = test_data["ts_10_7_norm"]

T, n_train = u_train_n.shape
n_test = u_test_n.shape[1]

N_trials = 30 # Number of Bayesian Optimiser trials

# Recreating reservoir with different functions to add the extra
hyperparameters
def create_base_reservoir(max_size, in_size=1, sparsity=1.0):
    W = np.random.rand(max_size, max_size) - 0.5
    # Zero out certain entries
    if sparsity < 1.0:
        mask = np.random.rand(max_size, max_size) < sparsity
        W *= mask

    np.fill_diagonal(W, 0.0)
    Win = np.random.rand(max_size, in_size) - 0.5

    eigvals = np.linalg.eigvals(W)
    base_radius = np.max(np.abs(eigvals)) # Spectral radius of W
```

```

    return W, Win, base_radius

# Added input scaling(sigma_in), reservoir size(N), spectral radius
def make_reservoir_from_base(W_base, Win_base, base_radius,
                             N, spectral_radius, sigma_in):
    W = W_base[:N, :N].copy()
    Win = Win_base[:N, :].copy()

    if base_radius > 0:
        W *= spectral_radius / base_radius

    Win *= sigma_in
    return W, Win

# Alpha as leaking rate
def run_reservoir(W, Win, u, leak_rate=1.0):
    """Leaky ESN state update."""
    N = W.shape[0]
    T = u.shape[0]
    x = np.zeros(N)
    all_x = np.zeros((T, N))

    alpha = leak_rate

    for t in range(T):
        pre_act = W @ x + Win @ u[t, :]
        x = (1 - alpha) * x + alpha * np.tanh(pre_act)
        all_x[t, :] = x

    return all_x

# Added beta as regularisation
def train_ridge(X, Y, beta):
    N = X.shape[1]
    A = X.T @ X + beta * np.eye(N)
    B = X.T @ Y
    Wout = np.linalg.solve(A, B)
    return Wout

# Generates predicted vorticity at (10,7)
def predict_reservoir(W, Win, Wout, u, leak_rate=1.0):
    N = W.shape[0]
    T = u.shape[0]
    x = np.zeros(N)
    alpha = leak_rate

    n_out = Wout.shape[1]
    y_pred = np.zeros((T, n_out))

```

```

    for t in range(T):
        pre_act = W @ x + Win @ u[t, :]
        x = (1 - alpha) * x + alpha * np.tanh(pre_act)
        y_pred[t, :] = Wout.T @ x

    return y_pred

# Used to find best hyperparameter combination by using minimum MSE
def mse(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)

# Entire hyperparameter space for tuning
search_space = {
    "N": (100, 600), # int, reservoir size
    "rho": (0.6, 1.3), # float, spectral radius
    "sigma_in": (0.1, 2.0), # float, input scaling
    "log10_beta": (-8, -2), # float, log10(regularisation)
    "alpha": (0.3, 1.0), # float, leaking rate
    "washout": (0, 60), # int, washout length
}

max_N = 600
sparsity = 1.0

W_base, Win_base, base_radius = create_base_reservoir(
    max_size=max_N,
    in_size=1,
    sparsity=sparsity
)

# Prebuild lists for convenience
u_train_list = [u_train_n[:, i][:, np.newaxis] for i in range(n_train)]
y_train_list = [y_train_n[:, i] for i in range(n_train)]

# Obtains random combination of hyperparameters
def sample_hyperparameters():
    N_min, N_max = search_space["N"]
    N = np.random.randint(N_min, N_max + 1)

    rho_min, rho_max = search_space["rho"]
    rho = np.random.uniform(rho_min, rho_max)

    sig_min, sig_max = search_space["sigma_in"]
    sigma_in = np.random.uniform(sig_min, sig_max)

    logb_min, logb_max = search_space["log10_beta"]
    log10_beta = np.random.uniform(logb_min, logb_max)
    beta = 10 ** log10_beta

```

```

a_min, a_max = search_space["alpha"]
alpha = np.random.uniform(a_min, a_max)

w_min, w_max = search_space["washout"]
washout = int(np.random.randint(w_min, w_max + 1))
washout = max(0, min(washout, T - 1)) # safety

return N, rho, sigma_in, beta, alpha, washout

def objective_random(N, rho, sigma_in, beta, alpha, washout,
trial_idx):

    # Build reservoir
    W, Win = make_reservoir_from_base(
        W_base, Win_base, base_radius,
        N=N,
        spectral_radius=rho,
        sigma_in=sigma_in,
    )

    # Build design matrix
    X_list = []
    Y_list = []

    for i in range(n_train):
        u_seq = u_train_list[i]
        x_seq = run_reservoir(W, Win, u_seq, leak_rate=alpha)
        X_list.append(x_seq[washout:, :])
        Y_list.append(y_train_list[i][washout:, np.newaxis])

    X_train = np.vstack(X_list)
    Y_train = np.vstack(Y_list)

    # Train readout
    Wout = train_ridge(X_train, Y_train, beta=beta)

    # Compute training MSE averaged over simulations
    mse_sims = []
    for i in range(n_train):
        u_seq = u_train_list[i]
        y_true = y_train_list[i]
        y_pred = predict_reservoir(W, Win, Wout, u_seq,
leak_rate=alpha)[: , 0]
        mse_sims.append(mse(y_true, y_pred))

    mean_train_mse = np.mean(mse_sims)

    # Progress print

```

```

print(
    f"[{trial_idx+1}/{N_trials}] "
    f"N={N}, rho={rho:.3f}, sigma_in={sigma_in:.3f}, "
    f"beta={beta:.1e}, alpha={alpha:.3f}, "
    f"washout={washout} -> train MSE={mean_train_mse:.3f}"
)

return mean_train_mse

# Random search optimisation
best_mse = np.inf
best_params = None

for trial in range(N_trials):
    N, rho, sigma_in, beta, alpha, washout = sample_hyperparameters()
    mean_train_mse = objective_random(
        N, rho, sigma_in, beta, alpha, washout, trial_idx=trial
    )

    if mean_train_mse < best_mse:
        best_mse = mean_train_mse
        best_params = (N, rho, sigma_in, beta, alpha, washout)

print(f"\nLowest training MSE after random search: {best_mse:.3g}")

best_N, best_rho, best_sigma_in, best_beta, best_alpha, best_washout =
best_params
print("Best hyperparameters found:")
print(f"  N      = {best_N}")
print(f"  rho    = {best_rho:.3g}")
print(f"  sigma_in = {best_sigma_in:.3g}")
print(f"  beta    = {best_beta:.3g}")
print(f"  alpha   = {best_alpha:.3g}")
print(f"  washout = {best_washout}")

# Training ESN with best hyperparameter combination
W_best, Win_best = make_reservoir_from_base(
    W_base, Win_base, base_radius,
    N=best_N,
    spectral_radius=best_rho,
    sigma_in=best_sigma_in,
)

# Rebuild training design matrix
X_list = []
Y_list = []

for i in range(n_train):

```

```

    u_seq = u_train_list[i]
    x_seq = run_reservoir(W_best, Win_best, u_seq,
leak_rate=best_alpha)
    X_list.append(x_seq[best_washout:, :])
    Y_list.append(y_train_list[i][best_washout:, np.newaxis])

X_train_best = np.vstack(X_list)
Y_train_best = np.vstack(Y_list)

Wout_best = train_ridge(X_train_best, Y_train_best, beta=best_beta)

# Training RMSE with tuned ESN
train_mse_sims = []
for i in range(n_train):
    u_seq = u_train_list[i]
    y_true = y_train_list[i]
    y_pred = predict_reservoir(
        W_best, Win_best, Wout_best, u_seq, leak_rate=best_alpha
   )[: , 0]
    train_mse_sims.append(mse(y_true, y_pred))

final_train_mse = np.mean(train_mse_sims)
print(f"\nNormalised training MSE with tuned ESN:
{final_train_mse:.3f}")

# -----
# 5. Evaluate on test set
# -----
u_test_list = [u_test_n[:, j][:, np.newaxis] for j in range(n_test)]
y_test_list = [y_test_n[:, j] for j in range(n_test)]

test_mse_sims = []
for j in range(n_test):
    u_seq = u_test_list[j]
    y_true = y_test_list[j]
    y_pred = predict_reservoir(
        W_best, Win_best, Wout_best, u_seq, leak_rate=best_alpha
   )[: , 0]
    test_mse_sims.append(mse(y_true, y_pred))

final_test_mse = np.mean(test_mse_sims)
print(f"Normalised test MSE with tuned ESN: {final_test_mse:.3f}")

ex_train = 0
u_tr = u_train_list[ex_train]
y_tr_true_n = y_train_list[ex_train]
y_tr_pred_n = predict_reservoir(
    W_best, Win_best, Wout_best, u_tr, leak_rate=best_alpha

```

```

)[: , 0]

y_tr_true = y_tr_true_n * std_ts_10_7 + mu_ts_10_7
y_tr_pred = y_tr_pred_n * std_ts_10_7 + mu_ts_10_7

ex_test = 0
u_te = u_test_list[ex_test]
y_te_true_n = y_test_list[ex_test]
y_te_pred_n = predict_reservoir(
    W_best, Win_best, Wout_best, u_te, leak_rate=best_alpha
)[: , 0]

y_te_true = y_te_true_n * std_ts_10_7 + mu_ts_10_7
y_te_pred = y_te_pred_n * std_ts_10_7 + mu_ts_10_7

fig, axs = plt.subplots(1, 2, figsize=(12, 4), sharey=True)

# Train and test plots
axs[0].plot(y_tr_true, label="True (train)")
axs[0].plot(y_tr_pred, label="ESN prediction (train)")
axs[0].set_xlabel("Time step")
axs[0].set_ylabel("Vorticity at (10,7)")
axs[0].set_title("Tuned ESN on Training Data")
axs[0].legend()

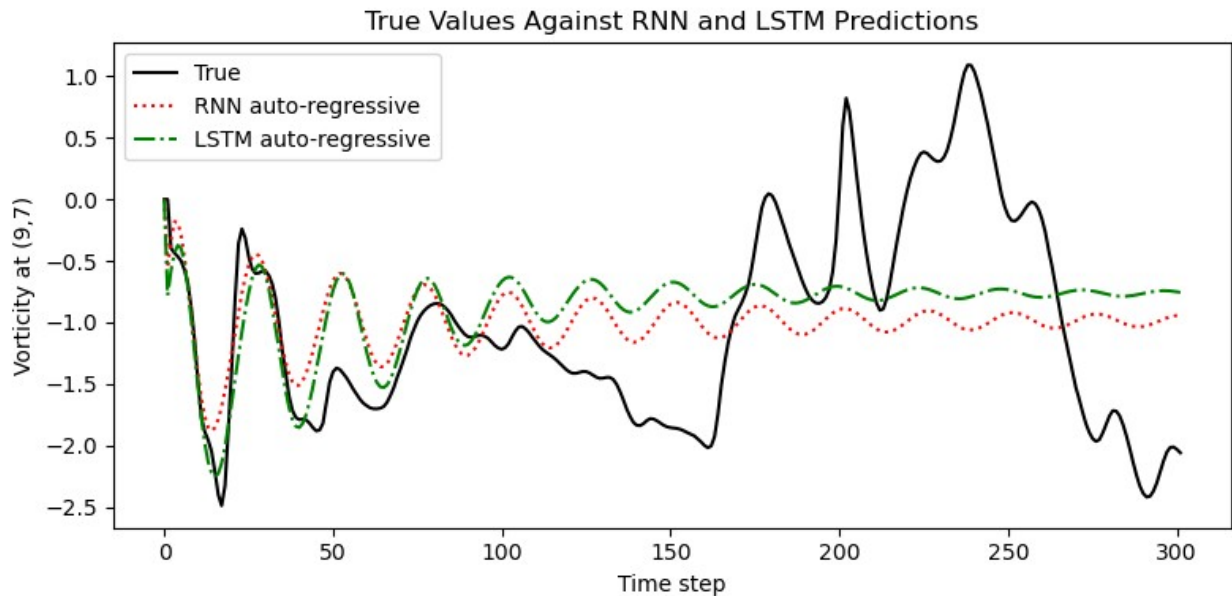
axs[1].plot(y_te_true, label="True (test)")
axs[1].plot(y_te_pred, label="ESN prediction (test)")
axs[1].set_xlabel("Time step")
axs[1].set_title("Tuned ESN on Test Data")
axs[1].legend()

plt.tight_layout()
plt.show()

```

Results:

Time: 3 mins 44.9 seconds



Lowest training MSE after random search: 0.253

Best hyperparameters found:

$N = 525$

$\rho = 1.1$

$\sigma_{in} = 0.154$

$\beta = 6.79e-05$

$\alpha = 0.378$

washout = 15

Normalised training MSE with tuned ESN : 0.253

Normalised Test MSE with tuned ESN : 0.323

The hyperparameters chosen are: the reservoir size, spectral radius, input scaling, leaking rate and regularisation. Random optimisation is used to tune the ESN hyperparameters since it is simple to implement and can explore a wide range of hyperparameter combinations without any assumptions. Each trial is independent of each other, which reduces the risk of overfitting since it is unaffected by noisy trials.

The most optimal combination after 30 trials was: $N = 525$, $\rho = 1.1$, $\sigma_{in} = 0.154$, $\beta = 6.79 \times 10^{-5}$, $\alpha = 0.378$ and washout length of 15. This hyperparameter combination gave a normalised MSE for the test error of 0.253. This is much lower than the pre-optimisation value of 3.962, showing that the choice of hyperparameters has a significant impact on ESN accuracy.

To further improve the modelling, the number of trials could be increased since this allows further exploration of the hyperparameter space. This could lead to better hyperparameter

choices and could improve generalisation, at the cost of increased computational time. Another way of improving the generalisation could be by adding noise into the inputs during training. This allows the model to become less sensitive to small fluctuations in the input signal and will be more accurate in replicating the true values.

Recurrent Neural Networks (RNNs)

Question 1

```
# For faster computational speed
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Vorticity at (9,7) [time, sim]
train_vort_9_7 = train_data["ts_9_7_norm"]
test_vort_9_7 = test_data["ts_9_7_norm"]

# Inputs for predictions
train_Re_norm = train_data["Re_norm"]
test_Re_norm = test_data["Re_norm"]
train_U_norm = train_data["U_norm"]
test_U_norm = test_data["U_norm"]

# Concatenate to size N
vort_norm = np.concatenate([train_vort_9_7, test_vort_9_7], axis=1)
Re_norm = np.concatenate([train_Re_norm, test_Re_norm], axis=0)
U_norm = np.concatenate([train_U_norm, test_U_norm], axis=0)

T, N = vort_norm.shape
n_train = train_vort_9_7.shape[1]
n_test = test_vort_9_7.shape[1]

# Index arrays within the concatenated representation
train_idx = np.arange(n_train)
test_idx = np.arange(n_train, n_train + n_test)

# Convert to torch so can be used later with torch tensors
train_idx_t = torch.tensor(train_idx, dtype=torch.long, device=device)
test_idx_t = torch.tensor(test_idx, dtype=torch.long, device=device)

vort_torch = torch.tensor(vort_norm, dtype=torch.float32,
device=device)
Re_torch = torch.tensor(Re_norm, dtype=torch.float32,
device=device)
U_torch = torch.tensor(U_norm, dtype=torch.float32,
device=device)

# Creating teacher forcing prediction
def build_teacher_forcing_sequences(vort_array, Re_array, U_array):
```

```

# Labelling input vectors and target outputs
T, N = vort_array.shape
X_list, y_list = [], []

# Inputs
for i in range(N):
    re = Re_array[i]
    u = U_array[i]
    v0 = vort_array[0, i]
    prev = vort_array[:-1, i] # Takes also previous true
vorticity as input

    x_i = np.stack(
        [
            np.full(T - 1, re, dtype=np.float32),
            np.full(T - 1, u, dtype=np.float32),
            np.full(T - 1, v0, dtype=np.float32),
            prev.astype(np.float32),
        ],
        axis=-1,
    )
    # Output target is true vorticity
    y_i = vort_array[1:, i:i+1].astype(np.float32)
    X_list.append(x_i)
    y_list.append(y_i)

X = np.stack(X_list, axis=0) # [N, T-1, 4]
y = np.stack(y_list, axis=0) # [N, T-1, 1]
return X, y

X_tf_np, y_tf_np = build_teacher_forcing_sequences(vort_norm, Re_norm,
U_norm)

X_tf = torch.tensor(X_tf_np, dtype=torch.float32, device=device)
y_tf = torch.tensor(y_tf_np, dtype=torch.float32, device=device)

# Train/test split in this combined indexing
X_tf_train = X_tf[train_idx_t]
y_tf_train = y_tf[train_idx_t]
X_tf_test = X_tf[test_idx_t]
y_tf_test = y_tf[test_idx_t]

# Standard RNN model classification
class SimpleRNN(nn.Module):
    def __init__(self, input_size, hidden_size=64, num_layers=1):
        super().__init__()
        self.rnn = nn.RNN(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=num_layers,

```

```

        batch_first=True,
    )
    self.fc = nn.Linear(hidden_size, 1)

# Describes how input flows through network to produce output
    def forward(self, x, h0=None):
        out, h = self.rnn(x, h0)
        out = self.fc(out)
        return out, h

# Parameters for RNN class
input_size = 4
hidden_size = 64
num_layers = 1

# Sets up loss function, model and optimiser
model_tf = SimpleRNN(input_size, hidden_size, num_layers).to(device)
criterion = nn.MSELoss()
optimizer_tf = optim.Adam(model_tf.parameters(), lr=1e-3)

epochs = 200
loss_history_tf = []

for epoch in range(epochs):
    model_tf.train()
    optimizer_tf.zero_grad()

    pred, _ = model_tf(X_tf_train) # [n_train, T-1, 1]
    loss = criterion(pred, y_tf_train)

    loss.backward()

    # Clipping prevents exploding gradients
    nn.utils.clip_grad_norm_(model_tf.parameters(), 1.0)
    optimizer_tf.step()

    loss_history_tf.append(loss.item())

# Training auto-regressive model
model_ar = SimpleRNN(input_size, hidden_size, num_layers).to(device)
optimizer_ar = optim.Adam(model_ar.parameters(), lr=5e-4)

loss_history_ar = []

for epoch in range(epochs):
    model_ar.train()
    optimizer_ar.zero_grad()

    batch_idx = train_idx_t
    n_batch = batch_idx.shape[0]

```

```

re_batch = Re_torch[batch_idx]
u_batch = U_torch[batch_idx]
v0_batch = vort_torch[0, batch_idx]

h = torch.zeros(num_layers, n_batch, hidden_size, device=device)
last_v = v0_batch # Set as true initial vorticity at t = 0

loss_epoch = 0.0

# At time t, constructing input
for t in range(1, T):
    x_t = torch.stack([re_batch, u_batch, v0_batch, last_v], dim=-
1)
    x_t = x_t.unsqueeze(1)

    out_t, h = model_ar(x_t, h)
    pred_v = out_t.squeeze(-1).squeeze(-1)

    true_v = vort_torch[t, batch_idx]
    loss_epoch = loss_epoch + criterion(pred_v, true_v)

    last_v = pred_v # Uses predicted vorticity

loss_epoch = loss_epoch / (T - 1)
loss_epoch.backward()
nn.utils.clip_grad_norm_(model_ar.parameters(), 1.0)
optimizer_ar.step()

loss_history_ar.append(loss_epoch.item())

# Function uses trained RNN over all timesteps for a given simulation
def rollout_autoregressive(model, Re_arr, U_arr, vort_arr,
sim_indices):
    model.eval()
    with torch.no_grad():
        batch_idx = sim_indices
        n_batch = batch_idx.shape[0]

        re_batch = Re_arr[batch_idx]
        u_batch = U_arr[batch_idx]
        v0_batch = vort_arr[0, batch_idx]

        h = torch.zeros(num_layers, n_batch, hidden_size,
device=device)
        last_v = v0_batch

        preds = [v0_batch.unsqueeze(1)]

        for t in range(1, T):
            x_t = torch.stack([re_batch, u_batch, v0_batch, last_v],

```

```

dim=-1)
    x_t = x_t.unsqueeze(1)

    out_t, h = model(x_t, h)
    pred_v = out_t.squeeze(-1).squeeze(-1)

    preds.append(pred_v.unsqueeze(1))
    last_v = pred_v

    preds = torch.cat(preds, dim=1)
    return preds          # Returns full predicted time series per
simulation

# Teacher forcing
model_tf.eval()
with torch.no_grad():
    pred_tf_test, _ = model_tf(X_tf_test)
    pred_tf_test = pred_tf_test.squeeze(-1)
    true_tf_test = y_tf_test.squeeze(-1)

# Both arrays assured to have length T
v0_test = vort_torch[0, test_idx_t].unsqueeze(1)

# Gives one step ahead predictions
pred_tf_full = torch.cat([v0_test, pred_tf_test], dim=1)
true_full    = vort_torch[:, test_idx_t].T

# Auto regressive
pred_ar_full = rollout_autoregressive(model_ar, Re_torch, U_torch,
                                     vort_torch, test_idx_t)

# Bring back to true values from normalised values
def unstandardise_v(v_norm):
    return v_norm * std_ts_9_7 + mu_ts_9_7

# Predicted and true vorticity values obtained
pred_tf_full_np = pred_tf_full.detach().cpu().numpy()
pred_ar_full_np = pred_ar_full.detach().cpu().numpy()
true_full_np    = true_full.detach().cpu().numpy()

pred_tf_phys = unstandardise_v(pred_tf_full_np)
pred_ar_phys = unstandardise_v(pred_ar_full_np)
true_phys    = unstandardise_v(true_full_np)

# Normalised MSE
mse_tf_t = np.mean((pred_tf_full_np - true_full_np) ** 2, axis=0)
mse_ar_t = np.mean((pred_ar_full_np - true_full_np) ** 2, axis=0)

# Average MSE for timesteps 1-100 and 201-300

```

```

mse_tf_1_100    = mse_tf_t[1:101].mean()
mse_tf_201_300 = mse_tf_t[201:301].mean()
mse_ar_1_100    = mse_ar_t[1:101].mean()
mse_ar_201_300 = mse_ar_t[201:301].mean()

print("\n Average normalised MSE on test set ")
print(f"Teacher forcing:  steps 1-100    MSE = {mse_tf_1_100:.2e}")
print(f"Teacher forcing:  steps 201-300 MSE = {mse_tf_201_300:.2e}")
print(f"Auto-regressive:  steps 1-100    MSE = {mse_ar_1_100:.2e}")
print(f"Auto-regressive:  steps 201-300 MSE = {mse_ar_201_300:.2e}")

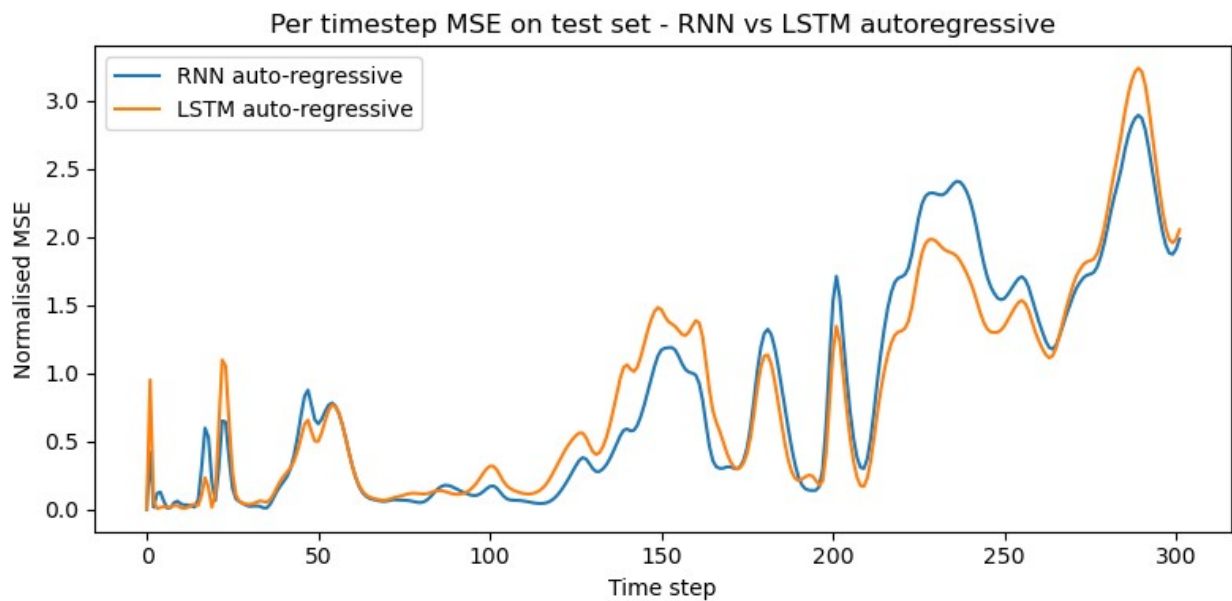
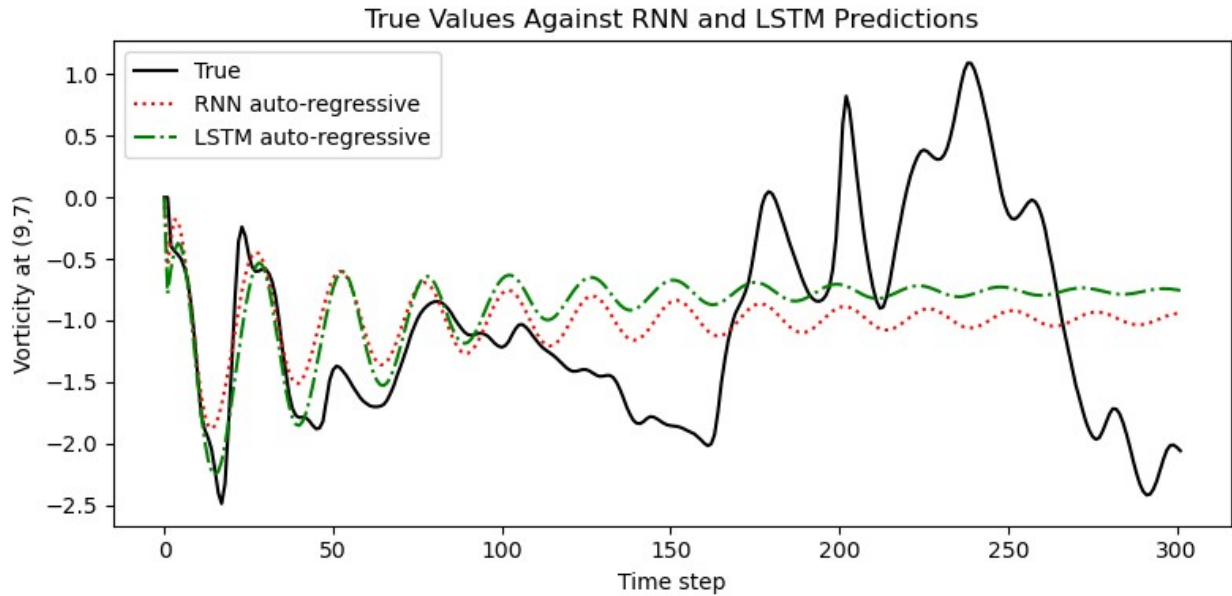
# Plotting graphs
t_array = np.arange(T) # Total timeframe

example_idx = 0
plt.figure(figsize=(8, 4))
plt.plot(t_array, true_phys[example_idx], "k-", label="True")
plt.plot(t_array, pred_tf_phys[example_idx], "b--", label="Teacher
forcing")
plt.plot(t_array, pred_ar_phys[example_idx], "r:", label="Auto-
regressive")
plt.xlabel("Time step")
plt.ylabel("Vorticity at (9,7)")
plt.legend()
plt.title("Testing simulation for True vs Predicted Values")
plt.tight_layout()
plt.show()

plt.figure(figsize=(8, 4))
plt.plot(t_array, mse_tf_t, label="Teacher forcing")
plt.plot(t_array, mse_ar_t, label="Auto-regressive")
plt.xlabel("Time step")
plt.ylabel("MSE")
plt.title("MSE for each timestep on test set")
plt.legend()
plt.tight_layout()
plt.show()

```

Results:



Time: 1 min 3.8 seconds

Teacher forcing: steps 1-100 $MSE = 0.0197$

Teacher forcing: steps 201-300 $MSE = 0.0170$

Auto-regressive: steps 1-100 $MSE = 0.224$

Auto-regressive: steps 201-300 $MSE = 1.78$

At timesteps 1-100, the teacher forcing method has an average normalised MSE of 0.0197 whereas the auto-regressive method has an average normalised MSE of 0.224. This shows that during the early part of the time series, the teacher forcing predictions are much more accurate than the auto-regressive predicted values. When looking at timesteps 201-300, the contrast is

even stronger. The auto-regressive model becomes highly inaccurate with a value of 1.78, whereas the teacher forcing method maintains the same order of accuracy with an average normalised MSE of 0.0170. Overall, the teacher forcing method proves to be more accurate during the early stages and the late stages of the time series compared to the auto regressive method.

The first time series plot above shows that the teacher forcing method curve stays close to the true vorticity across the entire timeframe, while the auto-regressive method initially starts close to the truth but drifts further away during the later time steps. This is further shown in the MSE per timestep plot which shows an initial peak for teacher forcing, which can be interpreted as a "warm up" period where the RNN is building its internal state before it has fully learnt the system's dynamics. This is shown by the rest of the graph where the error stays low with only one small peak at timestep 200. In contrast, the auto-regressive MSE generally shows an upwards trend against the timestep. This is due to the fact that each predicted value of vorticity is based off the previous predicted vorticity so previous errors while propagate into the later timesteps, causing the error to increase.

For this problem, teacher forcing is very effective at one-step predictions given that the input involves the previous true vorticity. This avoids feeding previous predictive errors back into the input, giving stable and accurate forecasts. The auto-regressive method is more realistic using previously predicted values as inputs. In this problem, its performance is poorer, as a result of the error accumulation from the nonlinear dynamics. This shows that the RNN is more suitable with one-step predictions.

Question 2

```
class SimpleLSTM(nn.Module):
    def __init__(self, input_size, hidden_size=64, num_layers=1):
        super().__init__()
        self.lstm = nn.LSTM(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True,
        )
        self.fc = nn.Linear(hidden_size, 1)

    # Same function as for RNN
    def forward(self, x, h_c=None):
        out, (h, c) = self.lstm(x, h_c)
        out = self.fc(out)
        return out, (h, c)

model_ar_lstm = SimpleLSTM(input_size, hidden_size,
                           num_layers).to(device)
optimizer_ar_lstm = optim.Adam(model_ar_lstm.parameters(), lr=5e-4)

# Recording all losses
loss_history_ar_lstm = []
```



```

for epoch in range(epochs):
    model_ar_lstm.train()
    optimizer_ar_lstm.zero_grad()

    batch_idx = train_idx_t
    n_batch = batch_idx.shape[0]

    re_batch = Re_torch[batch_idx]
    u_batch = U_torch[batch_idx]
    v0_batch = vort_torch[0, batch_idx]

    # LSTM hidden and cell state
    h = torch.zeros(num_layers, n_batch, hidden_size, device=device)
    c = torch.zeros(num_layers, n_batch, hidden_size, device=device)
    last_v = v0_batch

    loss_epoch = 0.0

    for t in range(1, T):
        # Input at time t: [Re, U, v_init, v_prev_pred]
        x_t = torch.stack([re_batch, u_batch, v0_batch, last_v], dim=-
1)
        x_t = x_t.unsqueeze(1)

        out_t, (h, c) = model_ar_lstm(x_t, (h, c))
        pred_v = out_t.squeeze(-1).squeeze(-1)

        true_v = vort_torch[t, batch_idx]
        loss_epoch = loss_epoch + criterion(pred_v, true_v)

        last_v = pred_v

    loss_epoch = loss_epoch / (T - 1)
    loss_epoch.backward()
    nn.utils.clip_grad_norm_(model_ar_lstm.parameters(), 1.0)
    optimizer_ar_lstm.step()

    loss_history_ar_lstm.append(loss_epoch.item())

# Same rollout method as for RNN
def rollout_autoregressive_lstm(model, Re_arr, U_arr, vort_arr,
sim_indices):
    model.eval()
    with torch.no_grad():
        batch_idx = sim_indices
        n_batch = batch_idx.shape[0]

        re_batch = Re_arr[batch_idx]
        u_batch = U_arr[batch_idx]

```

```

        v0_batch = vort_arr[0, batch_idx]

        h = torch.zeros(num_layers, n_batch, hidden_size,
device=device)
        c = torch.zeros(num_layers, n_batch, hidden_size,
device=device)
        last_v = v0_batch

        preds = [v0_batch.unsqueeze(1)]

        for t in range(1, T):
            x_t = torch.stack([re_batch, u_batch, v0_batch, last_v],
dim=-1)
            x_t = x_t.unsqueeze(1)

            out_t, (h, c) = model(x_t, (h, c))
            pred_v = out_t.squeeze(-1).squeeze(-1)

            preds.append(pred_v.unsqueeze(1))
            last_v = pred_v

        preds = torch.cat(preds, dim=1)
        return preds

# Predict on test set with LSTM autoregressive model
pred_ar_lstm_full = rollout_autoregressive_lstm(
    model_ar_lstm, Re_torch, U_torch, vort_torch, test_idx_t
)

# Obtaining predicted values and true values to get MSE
pred_ar_lstm_np = pred_ar_lstm_full.detach().cpu().numpy()
pred_ar_lstm_phys = unstandardise_v(pred_ar_lstm_np)

true_full_np = true_full.detach().cpu().numpy()

mse_ar_lstm_t = np.mean((pred_ar_lstm_np - true_full_np) ** 2, axis=0)

mse_ar_lstm_1_100 = mse_ar_lstm_t[1:101].mean()
mse_ar_lstm_201_300 = mse_ar_lstm_t[201:301].mean()

print("\n Average normalised MSE on test set")
print(f"LSTM auto-regressive: steps 1-100    MSE =
{mse_ar_lstm_1_100:.2e}")
print(f"LSTM auto-regressive: steps 201-300 MSE =
{mse_ar_lstm_201_300:.2e}")

example_idx = 0

# Comparison between RNN and LSTM for auto-regressive prediction
plt.figure(figsize=(8, 4))
plt.plot(t_array, true_phys[example_idx], "k-", label="True")

```

```

plt.plot(t_array, pred_ar_phys[example_idx], "r:", label="RNN auto-regressive")

plt.plot(t_array, pred_ar_lstm_phys[example_idx], "g-.", label="LSTM auto-regressive")

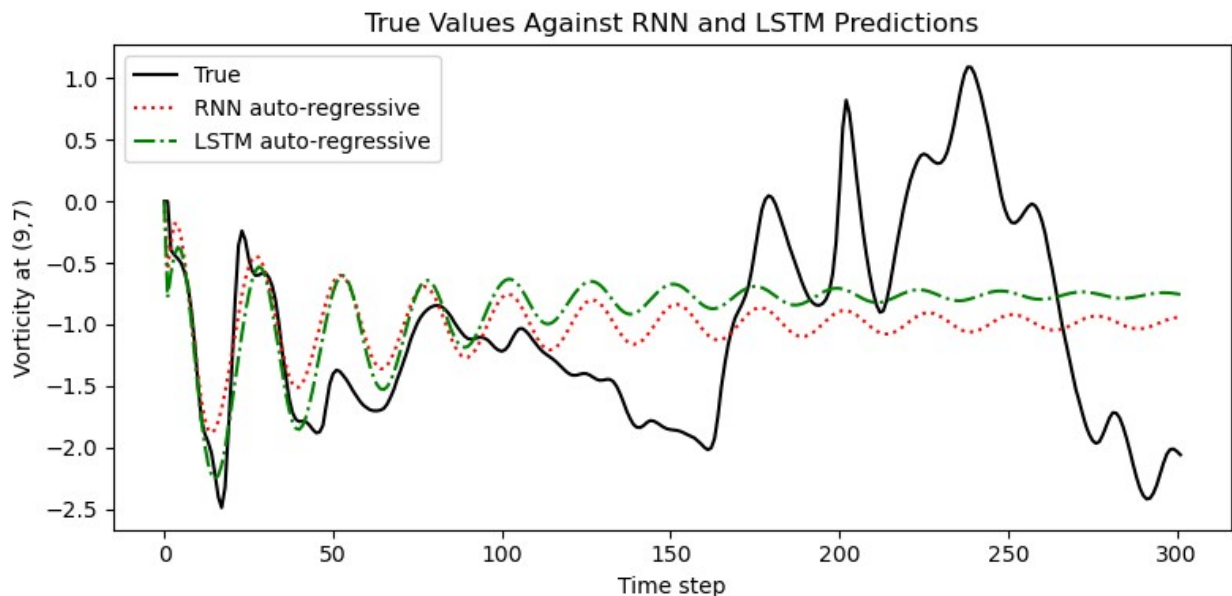
plt.xlabel("Time step")
plt.ylabel("Vorticity at (9,7)")
plt.title("True Values Against RNN and LSTM Predictions")
plt.legend()
plt.tight_layout()
plt.show()

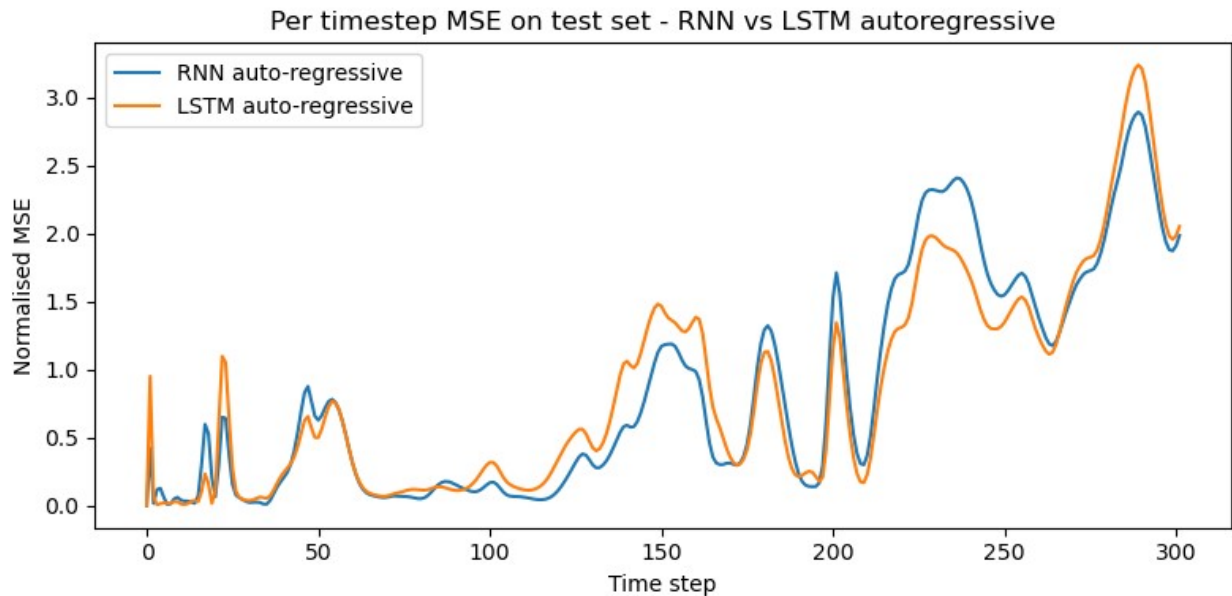
plt.figure(figsize=(8, 4))
plt.plot(t_array, mse_ar_t, label="RNN auto-regressive")
plt.plot(t_array, mse_ar_lstm_t, label="LSTM auto-regressive")
plt.xlabel("Time step")
plt.ylabel("Normalised MSE")
plt.title("Per timestep MSE on test set - RNN vs LSTM autoregressive")
plt.legend()
plt.tight_layout()
plt.show()

```

Results:

Time: 1 min 25.5 seconds





LSTM auto-regressive: steps 1-100 $MSE=0.231$

LSTM auto-regressive: steps 201-300 $MSE=1.64$

The auto-regressive RNN was replaced by an LSTM with the same input size, hidden size and number of layers. The first graph shows that both auto-regressive models exhibit the same path across the 300 timestep period. Both models are able to track the true vorticity to a reasonable extent showing an average $MSE=0.231$ for the first 100 timesteps. This is slightly less than the MSE for the RNN which shows that the training is slightly improved for the LSTM however for the later steps, there is a small increase in performance from the RNN to the LSTM. This demonstrates that for the later timesteps, the LSTM error accumulates in a similar manner as the RNN does. This is further shown in the second plot since over the entire time period, the two curves are very close to each other. Overall, this indicates that changing the model to an LSTM from an RNN does not meaningfully reduce the error at any point of the time period for this problem.

Encoder/Decoder Architectures

Question 1

```
# Part 1 - Build encoder
class Encoder(nn.Module):

    # Latent dim - number of latent states
    def __init__(self, latent_dim):
        super().__init__()

        # Multiple layers for stronger nonlinear mapping
        self.net = nn.Sequential(
            nn.Linear(386, 256),
            nn.ELU(True),
```

```

        nn.Linear(256, 128),
        nn.ELU(True),
        nn.Linear(128, latent_dim),
    )

    # Builds 386 dim input vector
    def forward(self, v_t_flat, Re, U):
        x = torch.cat(
            [v_t_flat, Re.unsqueeze(1), U.unsqueeze(1)],
            dim=1,
        )
        return self.net(x)

# Part 2 - Build decoder
class Decoder(nn.Module):
    def __init__(self, latent_dim):
        super().__init__()
        self.linear = nn.Linear(latent_dim, 384)

    def forward(self, z):
        return self.linear(z)

# Part 3 - Build linear time stepper
class LinearTimeStepper(nn.Module):
    def __init__(self, latent_dim, dt=0.1):
        super().__init__()
        self.dt = dt
        self.linear = nn.Linear(latent_dim, latent_dim)

    def forward(self, z_t):
        return self.linear(z_t)          # Computes z(t+dt)

```

Question 2

```

# Sigma, Re, U are already defined globally from
# Exploratory Data Analysis Question 1
K_max = 5
T_use = 100

# Restricting dataset to first 100 timesteps for train/test split
sigma_use = sigma[:, :, :T_use, :]
# Building training pairs for training
class VorticityPairDataset(Dataset):

    def __init__(self, sigma, Re, U, sim_indices, T_use=100, K_max=5):
        super().__init__()          # Storing all the variables
        self.sigma = sigma

```

```

self.Re          = Re
self.U           = U
self.sim_indices = sim_indices
self.T_use       = T_use
self.T_trans     = T_use - K_max

# Total no. of pairs over all the simulations
self.num_samples = len(sim_indices) * self.T_trans
self.K_max = K_max

def __len__(self):
    return self.num_samples

def __getitem__(self, idx):
    # Map flat index to (simulation, time)
    sim_idx_local = idx // self.T_trans      # which simulation
    t              = idx % self.T_trans      # which time t in
    0..T_use-2
    sim            = self.sim_indices[sim_idx_local]

    frames = []
    for j in range(self.K_max + 1):
        v = self.sigma[:, :, t + j, sim]
        frames.append(v.reshape(-1))

    v_seq_flat = np.stack(frames, axis=0)
    v_seq_flat = torch.from_numpy(v_seq_flat).float()

    # Scalars for that simulation
    Re_val = torch.tensor(self.Re[sim], dtype=torch.float32)
    U_val  = torch.tensor(self.U[sim],  dtype=torch.float32)

    return v_seq_flat, Re_val, U_val

# Use all simulations
train_sims = np.arange(N_sims) # or your train_idx if you want
train_dataset = VorticityPairDataset(
    sigma_use, Re, U,
    sim_indices=train_sims,
    T_use=T_use,
    K_max=K_max,
)
train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True)

```

Question 3

```
mse_loss = nn.MSELoss()
```

```

def compute_losses(encoder, decoder, stepper,
                   v_seq_flat, Re_val, U_val,
                   k_steps=1):

    v_seq_flat = v_seq_flat.to(device)      # (B, K_max+1, 384)
    Re_val      = Re_val.to(device)         # (B,)
    U_val       = U_val.to(device)

    v_t_flat    = v_seq_flat[:, 0, :]       # x(t)
    v_tp1_flat  = v_seq_flat[:, 1, :]       # x(t+1)
    v_tk_flat   = v_seq_flat[:, k_steps, :] # x(t+k)

    # Encode current and next fields
    z_t          = encoder(v_t_flat, Re_val, U_val)
    z_tp1_true   = encoder(v_tp1_flat, Re_val, U_val)

    # 1. Reconstruction error  $\Sigma r^2$ 
    v_t_hat      = decoder(z_t)
    loss_rec     = mse_loss(v_t_hat, v_t_flat)

    # 2. One step consistency error  $\Sigma l^2$ 
    z_tp1_pred   = stepper(z_t)
    loss_latent  = mse_loss(z_tp1_pred, z_tp1_true)

    # 3. K step rollout error  $\Sigma k^2$ 
    z_roll       = z_t.clone()
    for _ in range(k_steps):
        z_roll = stepper(z_roll)          # Apply for all k

    v_tk_hat     = decoder(z_roll)
    loss_rollout = mse_loss(v_tk_hat, v_tk_flat)

    return loss_rec, loss_latent, loss_rollout

# Function for training latent model for a value of latent dimension
def train_for_latent_dim(latent_dim,
                         num_epochs=40,
                         warmup_epochs=10,
                         base_lr=1e-3,
                         K_max=5):

    # Builds model
    encoder = Encoder(latent_dim).to(device)
    decoder = Decoder(latent_dim).to(device)
    stepper = LinearTimeStepper(latent_dim).to(device)

    params = list(encoder.parameters()) + \
              list(decoder.parameters()) + \
              list(stepper.parameters())

```

```

optimizer = optim.Adam(params, lr=base_lr)

# Sets up losses
history = {"total": [], "rec": [], "latent": [], "rollout": []}

for epoch in range(1, num_epochs + 1):

    # Reconstruction is only source of error at start
    if epoch <= warmup_epochs:
        current_k = 1
         $\lambda_r$ ,  $\lambda_1$ ,  $\lambda_k$  = 1.0, 0.0, 0.0      # reconstruction only
    else:
        # As training progresses, magnitude of weights decrease
        progress = (epoch - warmup_epochs) / (num_epochs -
warmup_epochs)

        # ramp k linearly from 1 up to K_max
        current_k = 1 + int(progress * (K_max - 1))
        current_k = max(1, min(current_k, K_max))

         $\lambda_r$  = 1.0

        # Weight adjustment as time progresses
         $\lambda_1$  = 0.1 + 0.9 * progress
         $\lambda_k$  = 0.1 + 0.9 * progress

    # Initialise accumulators
    total_epoch_loss = 0.0
    rec_epoch_loss = 0.0
    latent_epoch_loss = 0.0
    rollout_epoch_loss = 0.0
    n = 0

    for v_seq_flat, Re_val, U_val in train_loader:

        optimizer.zero_grad()

        loss_rec, loss_latent, loss_rollout = compute_losses(
            encoder, decoder, stepper,
            v_seq_flat, Re_val, U_val,
            k_steps=current_k,
        )

        total_loss =  $\lambda_r$  * loss_rec +  $\lambda_1$  * loss_latent +  $\lambda_k$  *
loss_rollout
        total_loss.backward()
        optimizer.step()

        B = v_seq_flat.size(0)

```



```

        total_epoch_loss += total_loss.item() * B
        rec_epoch_loss += loss_rec.item() * B
        latent_epoch_loss += loss_latent.item() * B
        rollout_epoch_loss += loss_rollout.item() * B
        n += B

    # Average loss per sample
    history["total"].append(total_epoch_loss / n)
    history["rec"].append(rec_epoch_loss / n)
    history["latent"].append(latent_epoch_loss / n)
    history["rollout"].append(rollout_epoch_loss / n)

    if epoch % 5 == 0 or epoch == 1 or epoch == warmup_epochs:
        print(
            f"[latent={latent_dim}] Epoch {epoch:3d}/{num_epochs}
| "

            f"| k={current_k} "
            f"Total={history['total'][-1]:.3e} "
            f"Rec={history['rec'][-1]:.3e} "
            f"Latent={history['latent'][-1]:.3e} "
            f"Roll={history['rollout'][-1]:.3e}"
        )

    return encoder, decoder, stepper, history

# Loop over multiple latent dimensions
latent_dims_to_try = [4, 8, 16, 32]
all_histories = {}

for ld in latent_dims_to_try:
    print("\n-----")
    print(f"Training for latent_dim = {ld}")
    print("-----")
    enc, dec, step, hist = train_for_latent_dim(
        latent_dim=ld,
        num_epochs=40,
        warmup_epochs=10,
        base_lr=1e-3,
        K_max=5,
    )
    all_histories[ld] = hist

print(f"{'latent_dim':>12} | {'final_total_loss':>18}")
print("-" * 40)
for ld in latent_dims_to_try:
    final_total = all_histories[ld]["total"][-1]
    print(f"{ld:12d} | {final_total:18.2e}")

```

Results:

Time: 11 mins 13.1 seconds

Latent Dimension = 4, total loss = 0.132

Latent Dimension = 8, total loss = 0.0529

Latent Dimension = 16, total loss = 0.0211

Latent Dimension = 32, total loss = 0.00583

The loss in this code is done by using a weighted sum of the three sources of error which is calculated as : $L = \lambda_r \Sigma_r^2 + \lambda_1 \Sigma_1^2 + \lambda_k \Sigma_k^2$.

Σ_r^2 represents the reconstruction error between the true vorticity field and the decoded field. Σ_1^2 represents the one step consistency error, penalising the error between the latent state obtained by encoding $x(t+1)$ directly and the latent state obtained by taking a single step forward using the linear time stepper. Finally, Σ_k^2 is the k step rollout error, which compares the true field with the field reconstructed after applying the latent dynamics repeatedly for k steps from the initial encoded state. Each λ is the weight each individual error component holds. These weights are constantly varied as the training process occurs.

Using the results above, there is a clear trend showing that as the number of latent states increases, the total loss decreases. This is because as the number of states increases, the model is being compressed into less states so the reconstruction and the rollout errors get smaller, so the total loss decreases.

Question 4

The code uses a linear time stepper in the latent space, so the latent dynamics are restricted to the form: $z_{t+1} = A z_t + b$. This means the latent loss can only fit dynamics that are approximately linear in the latent coordinates. This is not feasible for complex flow behaviour hence the solution is to use a nonlinear time stepper. This will allow the latent dynamics to capture nonlinear behaviour while the loss functions stay the same. The approximations would be more accurate, especially over longer prediction timeframes.

Question 5

One issue with using more than 100 timesteps is that it could lead to overfitting. Increasing the number of training samples could lead to the model memorising specific trajectories of the training simulations instead of learning the dynamics of the system. It would lead to much lower training loss however on unseen data, the predictions would deviate much more from the true results, indicating poor test accuracy. Another issue is that it is more computationally expensive. The higher the timesteps used, the greater the number of input/output pairs per simulation, so each epoch would require many more encoder, time stepper and decoder evaluations. This leads to longer training times which may force smaller batch sizes, or reduce model complexity.

For these issues, using the first 100 timesteps is a compromise to provide enough information to the encoder, decoder and time stepper for the learning process, while keeping the training cost manageable and limiting the risk of overfitting.


```

# ramp k linearly from 1 up to K_max
current_k = 1 + int(progress * (K_max - 1))
current_k = max(1, min(current_k, K_max))

λr = 1.0
λl = 0.1 + 0.9 * progress          # latent consistency
λk = 0.1 + 0.9 * progress          # rollout

total_epoch_loss = 0.0
rec_epoch_loss   = 0.0
latent_epoch_loss = 0.0
rollout_epoch_loss = 0.0
n = 0

for v_seq_flat, Re_val, U_val in train_loader:
    optimizer.zero_grad()

    loss_rec, loss_latent, loss_rollout = compute_losses(
        encoder, decoder, stepper,
        v_seq_flat, Re_val, U_val,
        k_steps=current_k,
    )

    total_loss = λr * loss_rec + λl * loss_latent + λk *
loss_rollout
    total_loss.backward()
    optimizer.step()

    B = v_seq_flat.size(0)
    total_epoch_loss += total_loss.item() * B
    rec_epoch_loss   += loss_rec.item() * B
    latent_epoch_loss += loss_latent.item() * B
    rollout_epoch_loss += loss_rollout.item() * B
    n += B

    history["total"].append(total_epoch_loss / n)
    history["rec"].append(rec_epoch_loss / n)
    history["latent"].append(latent_epoch_loss / n)
    history["rollout"].append(rollout_epoch_loss / n)

    if epoch % 5 == 0 or epoch == 1 or epoch == warmup_epochs:
        print(
            f"[latent={latent_dim},
stepper={stepper_cls.__name__}] "
            f"Epoch {epoch:3d}/{num_epochs} | k={current_k} | "
            f"Total={history['total'][-1]:.3e} "
            f"Rec={history['rec'][-1]:.3e} "
            f"Latent={history['latent'][-1]:.3e} "

```

```

        f"Roll={history['rollout'][-1]:.3e}"
    )

    return encoder, decoder, stepper, history

latent_dim = 16

print("\n Linear stepper")
enc_lin, dec_lin, step_lin, hist_lin =
train_for_latent_dim_with_stepper(
    latent_dim=latent_dim,
    num_epochs=50,
    warmup_epochs=10,
    base_lr=1e-3,
    K_max=5,
    stepper_cls=LinearTimeStepper,
)
print("\n Nonlinear stepper")
enc_nl, dec_nl, step_nl, hist_nl = train_for_latent_dim_with_stepper(
    latent_dim=latent_dim,
    num_epochs=50,
    warmup_epochs=10,
    base_lr=1e-3,
    K_max=5,
    stepper_cls=NonlinearTimeStepper,
)

# Print final losses for both steppers
print("\n Final Losses ")
print(f"{'Model':>12} | {'Total':>10} | {'Rec':>10} | {'Latent':>10} | "
      f"{'Rollout':>10}")
print("-" * 70)

def last(hist, key):
    return hist[key][-1]

print(f"{'Linear':>12} | {last(hist_lin, 'total'):10.3e} | "
      f"{last(hist_lin, 'rec'):10.3e} | {last(hist_lin, 'latent'):10.3e} "
      | " "
      f"{last(hist_lin, 'rollout'):10.3e}")

print(f"{'Nonlinear':>12} | {last(hist_nl, 'total'):10.3e} | "
      f"{last(hist_nl, 'rec'):10.3e} | {last(hist_nl, 'latent'):10.3e} | "
      " "
      f"{last(hist_nl, 'rollout'):10.3e}")

epochs = np.arange(1, len(hist_lin["latent"]) + 1)

```

Results:

Time: 7 mins 17.6 seconds

Linear time stepper has a k-step rollout error of 0.01161 and a one-step consistency error of 0.0005142 with a total error of 0.02226.

Nonlinear time stepper has a reconstruction error of 0.008776 and a one-step consistency error of 0.0003432 with a total error of 0.01986.

To measure the impact of replacing the linear time stepper with a nonlinear one, the one step consistency and the rollout loss must be assessed, since both depend on how well the latent dynamics are modelled. The results show that switching to a nonlinear time stepper reduced both individual error components, indicating that the nonlinear stepper provides a better approximation of the latent dynamics. The linear map can only represent dynamics that are approximately linear in the latent coordinates, which is too restrictive for the nonlinear flow behaviour in these simulations. The nonlinear stepper avoids this constraint and allows the model to approximate the latent dynamics more effectively.